

GROUP 6 DAY.

S/ N	NAME	REGISTRATION NUMBER
1	KIBI DARIUS	25/U/BIO/01383/PD
2	OTIM AMOS	25/U/BIO/01416/PD
3	INNOCENT BRYAN VUNI	25/U/BIO/01372/PD
4	ODORA STEPHEN	25/U/BIO/01411/PD
5	KUSENA JIMMY	25/U/BIO/01389/PD
6	JAKIRA JUMA FAHAD	25/U/BIO/01374/PD

Transport Fare Management System

C Programming Assignment

Presentation Outline

What we will cover today

01

Problem Statement

What challenge are we solving?

02

System Features

Full feature overview

03

Fare Calculation

Rules, discounts & surcharges

04

Program Design

Data structures & function map

05

Design Decisions

Why we built it this way

06

Challenges & Fixes

Obstacles we overcame

07

Live Demo Highlights

Key screens from the program

08

Conclusion

Summary & future work

Problem Statement

The challenge this system addresses

The Problem

Public transport operators in Uganda currently manage passengers and fares manually or using basic spreadsheets. This approach is:

- Prone to calculation errors
- Unable to enforce capacity rules
- Inconsistent with discount eligibility
- Lacking revenue tracking

Register passengers

Name, type, distance, luggage

Calculate fares

Distance-based + discounts + surcharge

Manage records

Search, update, delete

Trip summary

Revenue, top payer, depart status

System Features

Everything the TFMS can do

1

Authentication

Username & password login
Max 3 attempts before logout

2

Register Passenger

Name · type · distance · luggage
Auto-calculates fare on entry

3

Search Records

By unique ID or partial
case-insensitive name search

4

Update Records

Edit any field; fare
automatically recalculated

5

Delete Records

Confirmation prompt;
array compacted after removal

6

Trip Summary

Total revenue · top payer
Fill % · departure status

Fare Calculation Rules

How every ticket price is computed

FARE FORMULA

Base Fare = Distance (km) × UGX 500

Discounted = Base Fare × (1 - discount rate)

Surcharge = MAX(0, luggage - 20 kg) × UGX 2,000

Final Fare = Discounted Fare + Surcharge

Passenger Type	Discount	100 km example (0 kg luggage)
Adult	0%	UGX 50,000
Student	15%	UGX 42,500
Child	20%	UGX 40,000
Elderly	10%	UGX 45,000

UGX
500

per km
base rate

20 kg

free luggage
allowance

30

maximum
passengers

60%

min fill to
depart

Program Design & Structure

Data structures and key functions

Passenger Struct

```
int    id;           // auto 1001++
```

```
char   name[50];
```

```
PassengerType type; // enum
```

```
double distance;    // km
```

```
double luggageWeight; // kg
```

```
double fareCharged; // UGX
```

```
MAX_CAPACITY = 30 | MAX_PASSENGERS = 50
```

<code>authenticate()</code>	Login with 3-attempt limit
<code>mainMenu()</code>	Continuous menu dispatch loop
<code>registerPassenger()</code>	Collect + validate + store
<code>calculateFare()</code>	Pure: discount + surcharge
<code>searchPassenger()</code>	By ID or partial name
<code>updatePassenger()</code>	Edit fields, recalc fare
<code>deletePassenger()</code>	Confirm + compact array
<code>tripSummary()</code>	Revenue + fill + depart status

Design Decisions & Justifications

Why we built it this way

A

Array-Based Storage

Simple struct array chosen over linked list – sufficient for 30-seat capacity and easier to index, iterate, and compact after deletion.

B

Auto-Increment IDs

IDs start at 1001 and never reset, even after deletion, preventing accidental reuse of freed IDs.

C

Input Buffer Flushing

`clearInputBuffer()` called after every `scanf()` to prevent stale newlines from corrupting subsequent `fgets()` reads.

D

Case-Insensitive Search

Stored names and search keys are lowercased into temporary buffers, improving usability without altering stored data.

E

Pure `calculateFare()`

No I/O inside the fare function – it only receives parameters and returns a value, making it reusable, testable, and easy to update.

Challenges Encountered

Problems we faced and how we solved them

Mixed Input Handling



Problem: Using `scanf()` and `fgets()` together caused newline characters to remain in the buffer, corrupting subsequent reads.



Fix: Implemented `clearInputBuffer()` – reads and discards characters until newline or EOF after every numeric input.

Array Compaction on Delete



Problem: After removing a passenger, gaps in the array caused incorrect indexing and potential data corruption.



Fix: Shift all elements left from the deleted index position, then decrement `passengerCount`.

Capacity Enforcement



Problem: Ensuring both the 30-seat physical limit and the 50-record array limit were enforced independently and clearly.



Fix: Two separate guard clauses at the start of `registerPassenger()` with distinct error messages for each limit.

Program Demo Highlights

Key screens from the running system

transport_fare - Login & Menu

Username: admin

Password: ****

[+] Login successful. Welcome, admin!

----- MAIN MENU -----

1. Register Passenger
2. Search Passenger
3. Update Details
4. Delete Record
5. Display All
6. Trip Summary
0. Exit

Choice: _

transport_fare - Register

Name: Jane Nakato

Type: 2 (Student - 15% off)

Distance: 80 km

Luggage: 25 kg

[+] Registered. ID: 1001

Fare Charged: UGX 46,000.00

transport_fare - Trip Summary

=== TRIP SUMMARY ===

Passengers : 22 / 30

Fill Level : 73.3%

Status : READY TO DEPART

--- Revenue ---

Total Revenue : UGX 1,245,000

Highest Fare : UGX 112,000

Conclusion

- Fully functional menu-driven C program
- Secure authentication with logout
- Complete CRUD passenger management
- Accurate fare calculation with discounts & surcharge
- Capacity enforcement and departure status
- Clean, readable, function-based architecture

Future Work

File persistence · GUI · Multi-vehicle support · Receipt printing

GROUP 6

Day Programme
C Programming

Thank You!

Questions welcome